

3. SETS IN PYTHON

SETS

Built-in data types used to store numerous items in a single variable.

Characteristics

- ❑ Unordered
- ❑ Unindexed
- ❑ No duplicate elements
- ❑ Mutable

Example:

```
# creating a Set
```

```
S = {202, 205, 204, 209, 207}
```



Set of Employee ID

CREATING A SETS

1. Using curly braces

Example:

```
int_set = {12, 6, 7, 9, 11, 10}
```

```
mixed_set = {12, 'quest', 7.2, 6e2}
```

2. Using the set() Function

Example:

```
# given list
```

```
int_list = [6, 8, 1, 3, 7, 10, 4]
```

```
# creating set using set() function
```

```
int_set = set(int_list)
```

CREATING A SETS

Creating an empty set

```
empty_set = set()
```

```
print( empty_set ) # set()
```

Note : Empty curly braces '{}' will make an empty dictionary in Python.

Creating a set with a single value.

```
single_value_set = {"apple"} # Correct way to make a set with "apple"
```

Accessing Elements of a Set

Sets are unordered and unindexed, we cannot access the elements by position.

Example:

```
set_one = {11, 17, 12, 5, 7, 8}
```

```
print("Given Set:", set_one)
```

```
# iterating through the set using for loop
```

```
print("Iterating through the Set:")
```

```
for num in set_one:
```

```
    print(num)
```

Adding Elements to the Set

1. **add():** This method is used to add a single element to the set.

Syntax: `set.add(element)`

Example:

```
subjects = {'physics', 'biology', 'chemistry'}
```

```
subjects.add('maths') # {'physics', 'biology', 'chemistry', 'maths'}
```

2. **update():** This method is used to add multiple elements to the set.

Syntax: `set.update(iterable)` #Iterable means a list, tuple or another set

```
subjects.update(['computer', 'english']) #{'physics', 'chemistry', 'english', 'biology', 'computer', 'maths'}
```

Removing Elements to the Set

1. **remove()** : to remove a specific element from the set. It will raise a `KeyError` if the element is not found.

Example :

```
subjects = {'physics', 'chemistry', 'english', 'biology', 'computer', 'maths'}  
subjects.remove('maths')
```

2. **discard()** :

to remove a specified element from the set; however, it does not raise any error if the element is not found.

Example:

```
subjects.discard('chemistry')
```

Removing Elements to the Set

3. pop () : to remove and returns a random element from the set.

Example:

```
subjects.pop()
```

4. clear () : to remove all the elements from the given set.

Example:

```
subjects.clear()
```


SET OPERATIONS

1. union()

combines all unique elements from two or more sets into a new set without duplicates.

Syntax: `set1.union(set2,set3, ....)`

Or

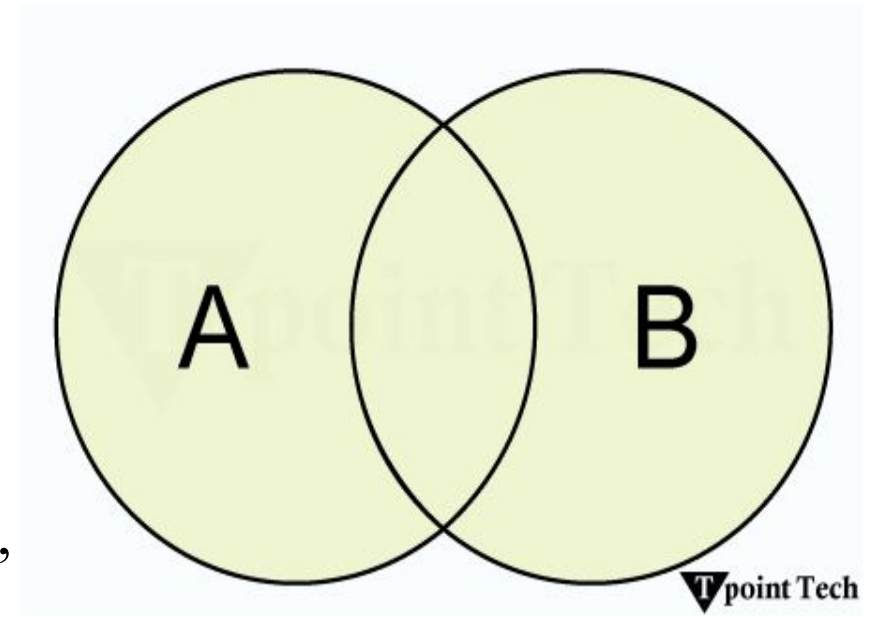
`set | set2 | set3 .....`

$A = \{2, 4, 5, 6\}$

$B = \{4, 6, 7, 8\}$

`result = A.union(B, C)` # Output: $\{2, 4, 5, 6, 7, 8, 9\}$,

`result2 = A | B | C` # Output: $\{2, 4, 5, 6, 7, 8, 9, 10\}$



SET OPERATIONS

2. intersection()

Returns a new set containing only the elements that are common to all the sets involved.

Syntax: `set1.intersection(set2, set3, ....)`

Or

`set1 & set2 & set3`

Example:

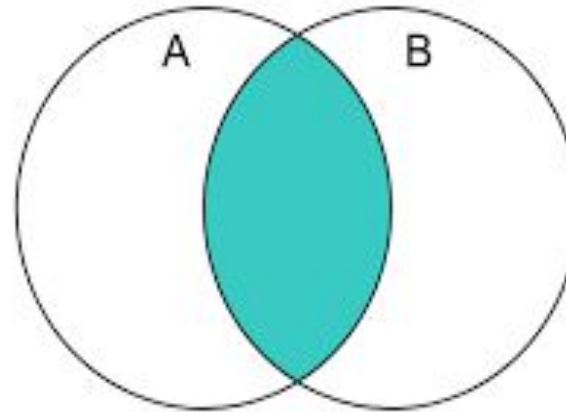
$A = \{2, 4, 5, 6\}$

$B = \{4, 6, 7, 8\}$

$C = \{4, 6, 8, 10\}$

`result = A.intersection(B, C) #output {4,6}`

`result2 = A & B & C #output {4,6}`



SET OPERATIONS

3. intersection_update ()

- To update an existing set by keeping only the elements that are common to itself and one or more other sets.
- This operation modifies the original set in place and does not return a new set.

Syntax : `set.intersection_update(set1, set2, ...)`

OR

`set &= set1 & set2 & ...`

Example:

`A = {1, 2, 3, 4}`

`B = {2, 3, 4, 5, 6}`

`C = {4, 5, 6, 9, 10}`

`A.intersection_update(B, C)`

`print(A) # Output: {4}`

SET OPERATIONS

4. difference ()

Difference of two sets A and B is defined as the set of all those elements which belongs to A, but do not belong to B and is denoted by A-B.

Syntax: `set1.difference(set2, set3, ...)`

OR

`set1 - set2 - set3`

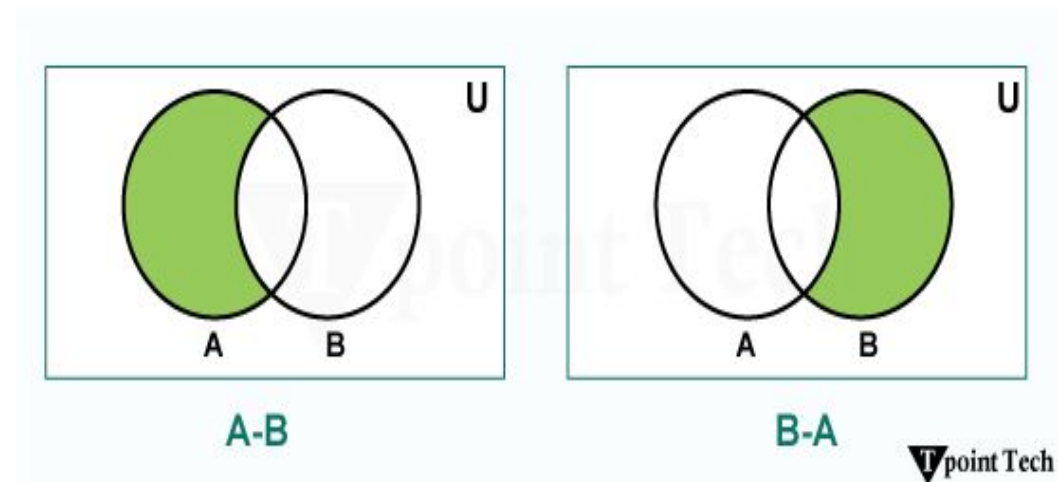
Example:

$A = \{1, 2, 3, 4\}$

$B = \{3, 4, 5, 6\}$

`diff1 = A.difference(B)` #output : $\{1, 2\}$

`Diff2 = A - B` #output : $\{1, 2\}$



SET OPERATIONS

5. difference_update ()

- to update a set by removing elements that are present in another set or multiple sets.
- It modifies the original set in place and does not return a new set.

Syntax:

```
set1.difference_update(set2, set3, ...)
```

Example:

```
A = {10, 20, 30, 40, 80}
```

```
B = {30, 40, 60, 80}
```

```
A.difference_update(B)
```

```
print(A) # Output: {10, 20}
```

SET OPERATIONS

6. symmetric_difference()

it provides all elements that are unique to each set, excluding the ones that they have in common.

Syntax: `set1.symmetric_difference(set2)` or `set1 ^ set2`

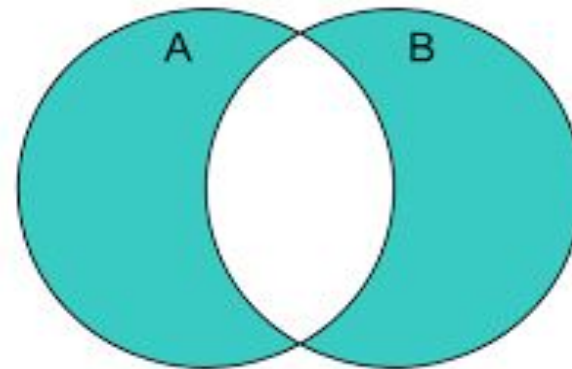
Example:

`A = {'a', 'b', 'c', 'd'}`

`B = {'c', 'd', 'e'}`

`result = A.symmetric_difference(B)` # Output: `{'a', 'b', 'e'}`

`print(A ^ B)` # Output: `{'a', 'b', 'e'}`



SET OPERATIONS

7. symmetric_difference_update()

It modifies the set in place by removing elements common to both sets and adding elements found in only one set.

Syntax: `set1.symmetric_difference_update(set2)` **OR** `set ^ = set2`

`A = {'a', 'c', 'd'}`

`B = {'c', 'd', 'e'}`

`A.symmetric_difference_update(B)`

`print(A)` # Output: `{'a', 'e'}`

SET METHODS

<code>issubset(other)</code>	Returns True if the set is a subset of other.
<code>issuperset(other)</code>	Returns True if the set is a superset of other.
<code>isdisjoint(other)</code>	Returns True if the set has no elements in common with other.

SET COMPREHENSION

set comprehension allows us to create sets in a concise and easy way.

Syntax: **{expression for variable in iterable}**

Example:

```
set_of_squares = {i**2 for i in range(6)}  #{0, 1, 4, 9, 16, 25}
```

```
set_of_cubes = {i**3 for i in range(6)}  #{0, 1, 64, 8, 27, 125}
```